



# **Lab Report**

## **Project- 01**

### **Submitted By:**

Syed Nazmus Sakib (233028312)  
Wasima Yeasmin Priya (233020112)

### **Submitted To:**

Farjana Alam Tofa

**Course Title:** Compiler Design Lab

**Course Code:** CSE320

**Date:** 23/02/2026

## Objective

The objective of this project is to build a lexical analyzer that tokenizes source code written in a subset of C. Also to recognize and classify tokens into categories: keywords, identifiers, numbers, symbols. And to learn to use the FLEX tool to generate the analyzer automatically from a set of pattern rules.

## Task Description

Lexical analysis is the first phase of a compiler. It reads the raw source code character by character and groups sequences of characters into meaningful units called tokens. These tokens are then passed to the next phase of the compiler (parsing). FLEX (Fast Lexical Analyzer Generator) is a tool that takes a .l file containing user-defined pattern rules written as regular expressions and generates a C source file (lex.yy.c) that implements the scanner. The generated file is compiled with GCC to produce the final executable analyzer.

The workflow used in this project:

```
lexer.l > flex lexer.l > lex.yy.c > gcc lex.yy.c -o lexer.exe
```

## Token Specification and Rules

Token Type: Keyword

Pattern Used: "int", "float", "double", "char", "if", "else", "while", "for", "return".

Description: Exact string matches for reserved words

Token Type: Identifier

Pattern Used: [a-zA-Z][a-zA-Z0-9]\*

Description: Starts with a letter, followed by letters or digits

Token Type: Integer

Pattern Used: [0-9]+

Description: One or more digit characters

Token Type: Float

Pattern Used: [0-9]+\.[0-9]+

Description: Digits, a dot, then more digits. This rule is placed before the integer rule to take priority

Token Type: Symbol

Pattern Used: `[(){};,]`

Description: Punctuation used for structure and separation

Whitespace and single-line comments (`//`) are silently skipped. Any unrecognized character is also skipped via a catch-all rule.

Each matched token is stored in an array using `add_unique()`, which prevents duplicate entries. At the end, all tokens are printed grouped by type on a single comma-separated line.

## FLEX Program Code

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TOKENS 1000
#define MAX_LEN    100

char keywords   [MAX_TOKENS][MAX_LEN]; int kw_count  = 0;
char identifiers[MAX_TOKENS][MAX_LEN]; int id_count  = 0;
char integers   [MAX_TOKENS][MAX_LEN]; int int_count = 0;
char floats     [MAX_TOKENS][MAX_LEN]; int flt_count = 0;
char symbols    [MAX_TOKENS][MAX_LEN]; int sym_count = 0;
char operators  [MAX_TOKENS][MAX_LEN]; int op_count  = 0;
char strings    [MAX_TOKENS][MAX_LEN]; int str_count = 0;

void add_unique(char arr[][MAX_LEN], int *count, const char
*val) {
    for (int i = 0; i < *count; i++)
        if (strcmp(arr[i], val) == 0) return;
    strncpy(arr[(*count)++], val, MAX_LEN - 1);
}

void print_group(const char *label, char arr[][MAX_LEN], int
count) {
    if (count == 0) return;
    printf("%-12s: ", label);
    for (int i = 0; i < count; i++) {
```

```

        printf("%s", arr[i]);
        if (i < count - 1) printf(", ");
    }
    printf("\n");
}
%}

%%

/* Keywords */
"int"      { add_unique(keywords, &kw_count, yytext); }
"float"    { add_unique(keywords, &kw_count, yytext); }
"double"   { add_unique(keywords, &kw_count, yytext); }
"char"     { add_unique(keywords, &kw_count, yytext); }
"if"       { add_unique(keywords, &kw_count, yytext); }
"else"     { add_unique(keywords, &kw_count, yytext); }
"while"    { add_unique(keywords, &kw_count, yytext); }
"for"      { add_unique(keywords, &kw_count, yytext); }
"return"   { add_unique(keywords, &kw_count, yytext); }

/* String literals */
\"[^\"]*\" { add_unique(strings, &str_count, yytext); }

/* Floating-point numbers (must come before integer rule) */
[0-9]+\.[0-9]+ { add_unique(floats, &flt_count, yytext); }

/* Integer numbers */
[0-9]+ { add_unique(integers, &int_count, yytext); }

/* Identifiers */
[a-zA-Z][a-zA-Z0-9]* { add_unique(identifiers, &id_count, yytext); }

/* Operators */
[+\-*/=><] { add_unique(operators, &op_count, yytext); }

/* Symbols */
[(){};:,] { add_unique(symbols, &sym_count, yytext); }

/* Single-line comments: ignore */
\\/\.* { /* skip */ }

/* Whitespace: ignore */
[ \t\n\r]+ { /* skip */ }

/* Catch-all for unrecognized characters */
. { /* skip unknown */ }

%%

```

```

int yywrap(void) {
    return 1;
}

int main(void) {
    yylex();
    printf("\nOutput:");
    print_group("KEYWORDS", keywords, kw_count);
    print_group("IDENTIFIERS", identifiers, id_count);
    print_group("INTEGERS", integers, int_count);
    print_group("FLOATS", floats, flt_count);
    print_group("SYMBOLS", symbols, sym_count);
    print_group("OPERATORS", operators, op_count);
    print_group("STRINGS", strings, str_count);
    return 0;
}

```

## Sample Input and Output

### Input:

```

int x;
float y;

if (x > 10) {
    return x;
} else {
    y = 3.14;
}

```

### Output:

```

Output:
KEYWORDS   : int, float, if, return, else
IDENTIFIERS : x, y
INTEGERS    : 10
FLOATS      : 3.14
SYMBOLS     : :, (, ), {, }
OPERATORS   : >, =

```

## **Discussion**

The lexical analyzer successfully identifies and classifies all required token types from the input source code. Keywords are matched using exact string literals, ensuring reserved words are never misclassified as identifiers. Identifiers follow the rule of starting with a letter and being followed by alphanumeric characters, which covers standard variable naming conventions. Numbers are handled with two separate rules floating-point is placed before integer in the .l file so that FLEX gives it higher priority, preventing 3.14 from being split into 3 and 14. Symbols and Operators cover all punctuation needed for program structure and expressions.